https://hjfy.top/arxiv/2602.02474

MemSkill：自演化智能体的学习与进化记忆技能

Haozhen Zhang[1] **Quanyu Long**[1] **Jianzhu Bao**[1] **Tao Feng**[2] **Weizhi Zhang**[3]
Haodong Yue[4] **Wenya Wang**[1]

Abstract

大多数大型语言模型(LLM)智能体的记忆系统依赖于一组少量静态的手动设计操作来提取记忆。这些固定流程将人类先验硬编码于存储内容与记忆更新方式之中，导致其在多样化的交互模式下表现僵化，并且在处理长期历史时效率低下。

为此，我们提出 **MemSkill**，将这些操作重新构想为可学习且可进化的记忆技能，即从交互迹中提取、整合和剪枝信息的结构化、可复用的常规流程。

受智能体技能设计哲学的启发，MemSkill 采用一个 *控制器*，学习选择一组相关技能；同时配备一个基于 LLM 的 *执行器*，生成由技能指导的记忆。

除了学习技能选择之外，MemSkill 还引入了一个 *设计师*，它周期性地审查那些所选技能产生错误或不完整记忆的困难案例，并通过提出改进方案和新增技能来演化技能集。

由此，MemSkill 构成一个闭环过程，不仅提升技能选择策略，还持续优化技能集本身。

在 LoCoMo、LongMemEval、HotpotQA 和 ALFWorld 上的实验表明，MemSkill 在性能上优于多个强基准方法，并在不同情景中展现出良好的泛化能力。

进一步的分析揭示了技能演化的机制，为构建更具适应性、自我演化的 LLM 智能体记忆管理提供了洞见。

代码已公开于 https://github.com/ViktorAxelsen/MemSkill

[1]Nanyang Technological University [2]University of Illinois Urbana-Champaign [3]University of Illinois Chicago [4]Tsinghua University. Correspondence to: Haozhen Zhang <haozhen001@e.ntu.edu.sg>, Wenya Wang <wangwy@ntu.edu.sg>.

Preprint. 2026 年 2 月 4 日.

1. 引言

随着大型语言模型（LLM）智能体展开更长时间、开放式交互，它们必须处理日益增长的历史记录，这些历史记录虽然至关重要，却难以有效利用，从而推动了对保留经验与维持连贯性的记忆机制的需求 (Hu et al., 2025)。这一需求催生了智能体记忆领域的快速发展，包括对过往交互进行总结和检索，或管理外部记忆存储等方法 (Kang et al., 2025; Chhikara et al., 2025; Packer et al., 2023; Xu et al., 2025)。然而，大多数现有方法仍依赖于静态的、人工设计的记忆机制，包括固定的操作原语（如添加/更新/删除/跳过）(Wang et al., 2025a; Yan et al., 2025)，以及决定存储内容、如何修改及何时清理的启发式模块 (Kang et al., 2025; Fang et al., 2025)。这类设计嵌入了强烈的人类先验假设，在多样化的交互模式下往往表现不佳，且随着历史记录的增长而难以有效缩放。

我们认为，这种表述从根本上限制了智能体记忆的适应性。与其将记忆视为固定操作或手工设计模块的输出，我们提出将记忆提取本身提升为一种可学习的抽象。具体而言，我们将记忆构建视为应用一组少量通用且可复用的记忆技能的结果：这些结构化

行为指定了交互迹应如何以及何时被变换为记忆，并随时间进行修订。这一视角揭示了先前流水线的一个关键瓶颈：它们将记忆行为硬编码进固定的程序工作流中，该工作流混合了启发式规则与基于大语言模型的提取和修订，导致在分布变化下表现脆弱 (Fang et al., 2025)。

从这一视角来看，理想的智能体记忆系统应满足三个特性。(i) *对人类先验的最小依赖*。记忆行为不应通过人工编码特定领域中值得记忆的内容 (Zhong et al., 2024)，而应由交互数据塑造，并随任务需求的演变而更新。(ii) *支持更大的提取粒度*。许多方法针对固定单元进行调优，例如每轮处理 (Fang et al., 2025)，当应用于更长跨度时性能会下降。一个实用的系统应在需要时能够以更大的提取粒度运行。(iii) *技能条件化的、可组合的记忆构建*。现有系统通常将记忆构建分解为专用模块 (Kang et al., 2025)。相比之下，我们更倾向于根据当前上下文选择并组合一组相关的技能，并在一次生成步骤中应用，从而实现记忆行为的灵活复用与演化。

基于上述观察，我们引入了 **MemSkill**，将记忆操作重新构想为一组可学习且可进化的记忆技能。MemSkill 维护一个共享的 *技能库*，其中每个技能捕获了一种可复用的方式，用于从交互文本中提取、整合或修订记忆（图 1 展示了记忆技能的结构化模板）。给定当前上下文，一个 *控制器*学习选择一组相关的技能，而基于大语言模型的 *执行器*则根据这些技能生成受技能引导的记忆，一次完成。这种技能条件化的形式不依赖于固定的提取单元，在处理长交互历史时可适用于不同的跨度长度。

至关重要的是，MemSkill 超越了学习如何使用一组固定技能的范畴。我们引入了一种闭环演化过程，该过程在学习使用当前技能库与演化技能库本身之间交替进行。具体而言，我们使用强化学习（RL）训练 *控制器*，以下游任务信号作为反馈来指导技能选择。周期性地，一个 *设计师*会汇总训练过程中产生的最困难案例，选取具有代表性的失败案例，并利用大语言模型（LLM）对现有技能进行优化并提出新的技能。每次演化步骤之后，控制器将在演化的技能库上继续训练，并通过额外的探索来促进对新引入技能的采纳。总体而言，这一过程逐步增强了技能选择策略和不断演化的技能库，朝着一种由交互数据驱动的更具适应性的记忆管理系统迈进。

在 LoCoMo、LongMemEval、HotpotQA 和 ALFWorld 上的实验表明，MemSkill 能够持续提升任务性能，并展现出良好的泛化能力。进一步的分析验证了关键组件的有效性，并展示了具有代表性的演化技能，为大语言模型智能体实现更自适应、自我演化的记忆管理提供了重要启示。

我们的贡献可概括如下。

- 我们提出 **MemSkill**，一种智能体记忆方法，将记忆操作表示为一个不断演进的技能库，并通过在选定的一组技能条件下对大语言模型进行条件控制，构建出由技能引导的记忆。
- 我们提出一种闭环最优化方法，结合强化学习进行技能选择，以及基于大语言模型引导的困难案例技能演化，实现技能库的持续优化，并朝着自演化智能体记忆系统迈出一步。
- 我们在 LoCoMo、LongMemEval、HotpotQA 和 ALFWorld 上评估了 MemSkill，结果表明其在各项任务中均持续优于基准模型，并展现出强大的泛化能力，为大语言模型智能体的自我演化记忆提供了重要启示。

2. 相关工作

2.1. 大语言模型智能体记忆系统

先前关于智能体记忆的工作主要集中在从交互历史中构建外部记忆，并利用这些记忆来支持下游推理与决策。典型的流水线会周期性地将关键信息提取到记忆存储中，针对新查询检索相关条目，并通过整合或剪枝来更新存储 (Kang et al., 2025; Zhong et al., 2024; Xu et al., 2025; Packer et al., 2023; Chhikara et al., 2025; Fang et al., 2025)。最近，基于学习的方法如 Memory-R1 (Yan et al., 2025) 和 Mem-α (Wang et al., 2025a) 利用下游任务信号，

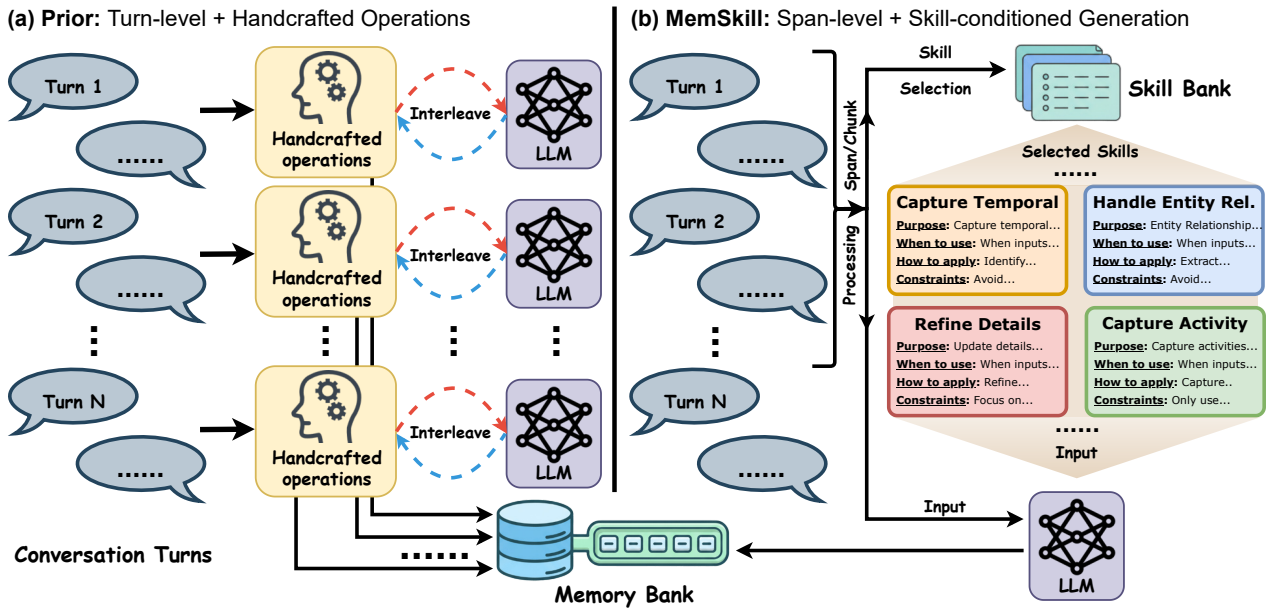

Figure 1. **(a) 先验的逐轮手工操作与 (b) MemSkill 的跨度级、技能条件生成之间的对比。**先验方法将手工设计的操作与大语言模型调用交替进行，逐轮增量式地提取和修正记忆，而 MemSkill 则从共享的技能库中选择一组少量技能，并在单次遍历中应用这些技能以生成受技能引导的记忆。

通过强化学习优化记忆管理。尽管取得了进展，记忆管理仍然主要由静态的、人工设计的提取、整合和剪枝规则所主导。

若干并发工作也探讨了智能体情景下的自演化记忆，但其根本上与我们的研究重点不同。Evo-Memory 提供了一个用于测试时记忆演化的流式基准和评估框架 (Wei et al., 2025)，而 MemEvolve 则在预定义的模块化设计空间内对记忆架构进行元优化 (Zhang et al., 2025)。相比之下，我们关注的是记忆技能本身的演化，使系统能够随着时间推移不断精炼和扩展其可复用的记忆操作。

2.2. 自演化大语言模型智能体

近期关于自演化大语言模型智能体的研究探讨了智能体如何在极少人工监督的情况下，通过与环境的交互经验实现自我提升。ExpeL (Zhao et al., 2024) 将轨迹提炼为可编辑的自然语言洞察，并检索相关经验以指导未来决策；而 EvolveR (Wu et al., 2025) 则形式化了一种经验生命周期，将交互过程整合为可复用的原则，并通过强化学习更新闭环完成演化。另一条研究路径则通过自对弈风格的课程设计降低对精心筛选数据的依赖：Absolute Zero Reasoner (Zhao et al., 2025) 采用代码执行器提供的可验证奖励，训练一个提议者与求解者；Multi-Agent Evolve (Chen et al., 2025) 将其扩展为基于大语言模型评估的提议者-求解者-评判者三元组；R-Zero (Huang et al., 2025) 则遵循类似的挑战者-求解者协同演化模式。除了课程设计之外，AgentEvolver (Zhai et al., 2025) 和 RAGEN (Wang et al., 2025b) 研究了多轮强化学习情景下的高效智能体学习动态与稳定性，而 ADAS (Hu et al., 2024) 与 AlphaEvolve (Novikov et al., 2025) 探索了智能体设计的自动化发现与演化改进。最后，Skill-Weaver (Zheng et al., 2025) 表明智能体能够发现并优化用于网络交互的可复用技能。相比之下，我们的研究重点在于自演化式的记忆技能，即智能体如何随时间构建与修订记忆。

3. 方法

在本节中，我们首先概述 MemSkill（第 3.1 节），然后详细说明 技能库（第 3.2 节）以及三个核心组件（控制器（第 3.3.1 节）、执行器（第 3.3.2 节）和 设

计者（第 3.4 节）），最后总结闭合环最优化过程，该过程在学习使用当前技能与从困难案例中演化技能库之间交替进行（第 3.5 节）。

3.1. 概述

如图 2 所示，我们提出了 **MemSkill**，该方法通过两个相互交织的过程优化智能体记忆。第一个过程 **学习如何使用给定的技能库**：控制器根据上下文选择一小部分技能，执行器应用这些技能以生成记忆更新。第二个过程 **改进技能库本身**：设计者在训练过程中，基于具有挑战性的案例周期性地修订现有技能并引入新技能。

为了将与特定迹相关的记忆与可复用的记忆管理知识解耦，MemSkill 维护两个独立的存储。记忆库是针对特定迹的，用于存储为每个训练迹（例如，一段长对话）构建的记忆。相比之下，技能库在所有迹之间共享，包含可复用的记忆技能。在训练过程中，控制器和执行器与特定迹交互以构建其记忆库，而设计者则在各阶段之间更新共享的技能库。这种交替过程逐步提升了技能选择策略以及用于记忆构建的技能库。

3.2. 技能库

如图 2 所示，一种 记忆技能将可重用的记忆操作作为结构化指导进行指定，包括其适用时机以及如何应用于当前上下文。具体而言，每种技能 $s \in \mathcal{S}$ 包含：(i) 一个用于技能表示和选择的简短 描述，以及 (ii) 一份详细的 内容规范，用于指导执行者如何完成记忆提取或修订。

我们从一组最小化的通用基本操作开始，以确保稳定且功能完备的初始化。具体而言，我们使用四个对应于正则记忆操作的基本技能来初始化技能库：Insert、Update、Delete 和 Skip。从这一最小集合出发，设计者逐步优化现有技能，并通过提出能够解决未覆盖失败模式的新技能来扩展技能库。（附录 B 详细描述了技能）

3.3. 学习使用记忆技巧

在本部分中，我们描述了 MemSkill 如何学习使用记忆技能，包括（i）技能选择策略和（ii）基于技能的记忆构建。

3.3.1. 控制器：技能选择策略

为了在技能库演进过程中实现有效的技能选择，我们引入了一个控制器，用于为当前上下文选择一组相关的记忆技能。在每个记忆构建步骤中，**我们在片段级别更新记忆**：将每次交互迹（例如，对话）分割成连续的文本片段，并按顺序处理；对于每个片段，控制器的选取基于（i）当前文本片段和（ii）检索到的现有记忆，而非逐轮操作。

为了保持与不断演化的可变大小技能库的兼容性，控制器通过测量当前状态表示与技能表示之间的语义距离来对每个技能进行评分，这自然支持技能集合的变化，同时仍能敏感地响应内存中已存储的内容。

状态表示。形式上，令 x_t 表示步骤 t 时的当前文本片段，令 $M_t = \{m_{t,1}, \ldots, m_{t,R}\}$ 为从当前轨迹的内存库中检索到的记忆。控制器将 (x_t, M_t) 编码为状态嵌入：

$$h_t = f_{\text{ctx}}(x_t, M_t). \tag{1}$$

技能表示。对于当前技能库中的每项技能 $s_i \in \mathcal{S}_t$，我们从其描述中计算出一个技能嵌入，因为描述提供了比完整技能内容更稳定且聚焦的语义信号。

$$u_i = f_{\text{skill}}(\text{desc}(s_i)). \tag{2}$$

请注意，我们对 f_{ctx} 和 f_{skill} 使用相同的嵌入模型，将上下文和技能描述映射到共享表示空间中以进行评分。

与不断演化的技能库的兼容性。与生成固定维度的动作头并绑定固定数量技能不同，控制器通过比较状态嵌入和技能嵌入来为每项技能打分：

$$z_{t,i} = h_t^\top u_i, \qquad p_\theta(i \mid h_t) = \text{softmax}(z_t)_i, \tag{3}$$

其中 $z_t \in \mathbb{R}^{|\mathcal{S}_t|}$ 会随着技能库的演变自动调整。

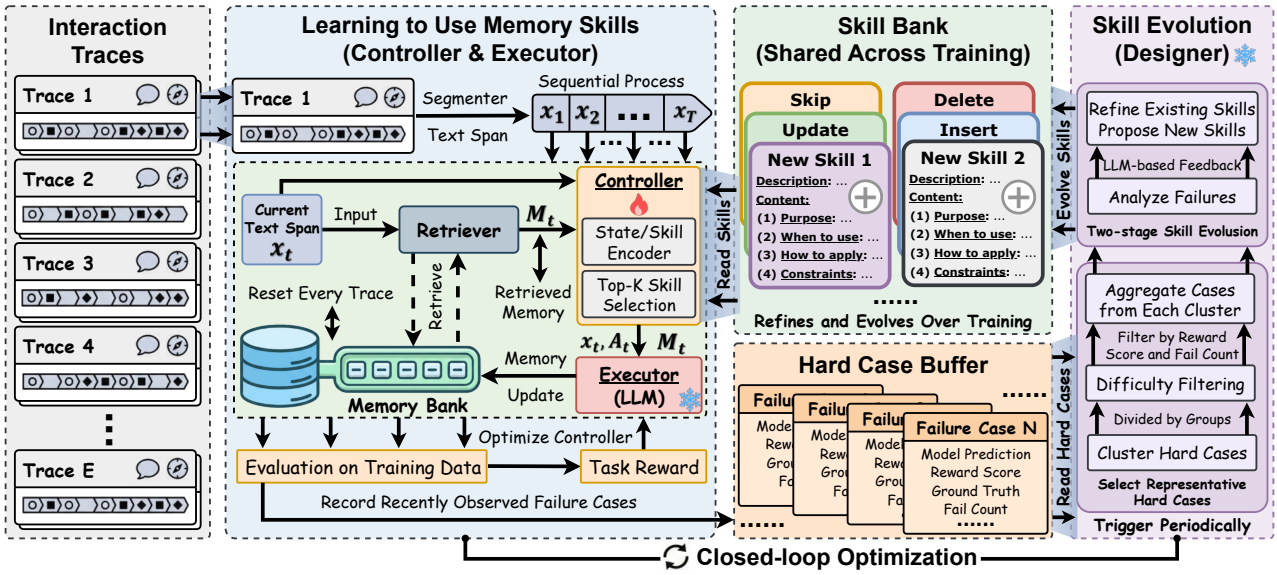

Figure 2. **MemSkill 架构概览**。给定一个交互迹，MemSkill 按段落逐段处理：控制器根据当前文本段落和检索到的记忆，从共享的技能库中选择 Top-K 个技能子集，随后由大语言模型执行器在一次遍历中应用所选技能，以更新与迹相关的记忆库。构建的记忆将针对依赖记忆的训练查询进行评估，以提供任务奖励来优化控制器，同时查询中心的失败情况会被记录到滑动的困难案例缓冲区中。周期性地，设计者挖掘具有代表性的困难案例，以改进现有技能并提出新技能，从而形成技能使用与技能演化的交替阶段。更多技能案例研究详见第 4.5 节及附录 B。

Top-K 技能选择。给定当前技能库 $\mathcal{S}_t$ 上的类别分布 $p_\theta(i \mid h_t)$，控制器以无放回的方式（例如通过 Gumbel-Top-K (Kool et al., 2019)）选出有序的 Top-K 技能集 $A_t = (a_{t,1}, \ldots, a_{t,K})$，仅将所选技能传递给执行器，从而保持技能上下文的简洁性和相关性。

3.3.2. 执行器：技能条件记忆提取

给定选定的技能 A_t，执行器（固定）通过将大语言模型（LLM）以（i）当前文本片段 x_t、(ii) 检索到的记忆项 M_t 以及（iii）选定的技能 A_t 作为条件，构建记忆更新。这模拟了智能体系统中的技能条件化推理，即提供一组相关的技能来指导当前上下文下的行为。

随后，执行器以结构化格式生成记忆更新，并解析后将其应用于更新迹的记忆库。

通过在同一文本片段上组合多个技能并一次性完成记忆提取，MemSkill 减少了每轮重复处理的开销，使记忆构建更易于扩展至长交互历史。

附录 C 详细说明了完整的执行器提示。

3.3.3. 控制器最优化

我们使用强化学习训练控制器，以下游任务性能作为其技能选择的反馈。对于每个训练迹，控制器会做出一系列 Top-K 项的选择，同时执行器逐步构建与该迹相关的内存库。构建完成后，我们在该迹的依赖内存的训练查询上评估生成的内存库，并将得到的任务性能作为奖励（例如 F1 值或成功概率）。

一个关键技术细节是，控制器的动作是一个不重复选取的前 Top-K 集，而非单一的离散动作。因此，我们计算在不重复选取过程下的联合对数概率 $\log \pi_\theta(A_t \mid s_t)$，并通过重要性加权和截断将其用于标准策略梯度风格的目标。具体而言，联合概率可表示为

$$\pi_\theta(A_t \mid s_t) = \prod_{j=1}^{K} \frac{p_\theta(a_{t,j} \mid s_t)}{1 - \sum_{\ell<j} p_\theta(a_{t,\ell} \mid s_t)}, \qquad (4)$$

当 $K = 1$ 时，退化为通常的单动作情形。附录 A.4 提供了实现细节。

3.4. 通过设计师反馈实现技能演化

除了学习从固定技能集合中进行选择外，MemSkill 还通过一个基于大语言模型的设计器（固定）在训练期间周期性地演化技能库。

困难案例缓冲区。在控制器训练过程中，我们维护一个滑动窗口缓冲区，用于记录最近观察到的挑战性案例。每个案例以查询为中心，记录查询及其真实标签和元数据（例如检索到的记忆和模型预测），以及诸如任务性能和当前已观察到的失败次数等汇总统计量。该缓冲区采用两种过期规则：若案例过于陈旧（超过最大训练步数间隔）则被移除，或当缓冲区达到其容量上限时，也会移除案例，从而在不无限增长的情况下跟踪近期的失败模式。

选择具有代表性的难例。为了使设计者更新聚焦于有影响力的失败案例，我们使用聚类方法（如 KMeans）将案例分组，这些组自然反映了不同的查询或错误类型。在每个簇内，我们通过一个难度得分来优先选择具有代表性的案例，该得分在任务性能较低且同一案例反复失败时升高。这生成了一组紧凑的高价值案例，用于技能演化，同时保留了不同错误类型之间的多样性。

两阶段技能演化。设计者以两个阶段更新技能库。首先，它利用大语言模型分析选定的困难案例，识别出缺失或错误指定的记忆行为。其次，基于该分析结果，提出对现有技能的具体修改建议，并引入新技能。此处我们保持设计者描述简洁，并在附录 C中提供提示细节。

值得注意的是，我们保留表现最佳的技能库快照，并在更新导致性能下降时进行回滚，当重复的设计者更新无法改善训练信号时采用早停。在每次演化步骤后，我们还会短暂增加探索力度，通过偏向选择新引入的技能来激励控制器尝试这些技能，从而促进其效用的高效学习。关于设计者的更多细节见附录 A.2。

3.5. 闭环最优化

MemSkill 在（i）学习如何选择并应用技能以构建记忆库，以及（ii）基于近期训练步骤中挖掘出的困难案例来演化技能库之间交替进行。每个周期开始时，控制器在当前技能库上进行训练，期间执行器构建记忆，系统积累具有挑战性的案例。随后，设计者使用具有代表性的困难案例更新技能库，如果更新导致性能退化，可选择回滚到之前的快照。下一周期在更新后的技能库上继续控制器训练，并增加探索力度，以促进新技能的早期使用。通过反复循环，MemSkill 逐步提升技能使用效率以及用于记忆构建的技能库质量。

4. 实验

4.1. 实验设置

数据集与基准. 我们在四个基准上评估 MemSkill：LoCoMo (Maharana et al., 2024)、LongMemEval (Wu et al., 2024)、HotpotQA (Yang et al., 2018) 以及 ALFWorld (Shridhar et al., 2020)，其中 HotpotQA 在第 4.4 节用于研究分布偏移下的技能迁移。其余三个基准覆盖了两种典型情景。(i) *对话型基准*包括 LoCoMo 和 LongMemEval，用于评估从长时、对话式交互历史中构建记忆的能力。针对这些数据集，我们报告 F1 得分（F1）和基于大语言模型的评判得分（L-J）。(ii) *具身交互任务*在 ALFWorld 上使用两个标准子集进行评估，即 ALF-Seen 和 ALF-Unseen，我们报告成功率（SR）和环境交互步数（#Steps）。具体的数据集划分见附录 A.1。

我们对比了 MemSkill 与多个强基准方法：(1) **No-Memory**，不使用外部记忆（或额外构建的上下文）直接作答；(2) Chain-of-Notes (**CoN**) (Yu et al., 2024)；(3) **ReadAgent** (Lee et al., 2024)；(4) **MemoryBank** (Zhong et al., 2024)；(5) **A-MEM** (Xu et al., 2025)；(6) **Mem0** (Chhikara et al., 2025)；(7) **LangMem** (LangChain, 2025)；以及 (8) **MemoryOS** (Kang et al., 2025)。总体而言，该设置涵盖了多种基准和基准方法，能够在多样化的情景下实现广泛且一致的比较。

Table 1. **LoCoMo、LongMemEval 和 ALFWorld 的主要对比结果。**

Model	Methods	Conversational Benchmarks					Embodied Interactive Tasks				
		LoCoMo		▲LongMemEval		Avg.	ALF-Seen		ALF-Unseen		Avg.
		F1	L-J	F1	L-J	L-J	SR	#Steps↓	SR	#Steps↓	SR
LLaMA3.3 70B-Instruct	No-Memory	-	-	-	-	-	17.14	43.74	20.15	42.99	18.65
	CoN	17.97	24.80	30.28	56.93	40.87	40.71	33.44	30.60	37.66	35.66
	ReadAgent	26.34	35.17	23.52	41.58	38.38	32.86	37.09	38.06	34.78	35.46
	MemoryBank	33.54	40.92	30.26	35.15	38.04	25.00	39.96	32.84	36.54	28.92
	A-MEM	35.60	46.34	25.86	38.12	42.23	24.29	40.51	28.36	38.83	26.33
	Mem0	10.18	33.01	29.94	45.54	39.28	32.86	36.47	32.09	37.32	32.48
	LangMem	25.97	29.14	15.79	21.00	25.07	37.86	34.39	35.07	35.70	36.47
	MemoryOS	38.68	44.59	14.19	36.50	40.55	15.71	43.74	14.18	44.54	14.95
	MemSkill	**38.78**	**50.96**	**31.65**	**59.41**	**55.19**	**47.86**	**30.88**	**47.01**	**30.43**	**47.44**
▲Qwen3-Next 80B-A3B-Instruct	No-Memory	-	-	-	-	-	18.57	42.48	26.12	39.35	22.35
	CoN	27.97	35.35	**28.34**	46.04	40.70	57.86	25.81	53.73	28.40	55.80
	ReadAgent	25.41	33.57	23.52	41.58	37.58	53.57	27.88	54.48	27.41	54.03
	MemoryBank	25.39	39.76	7.36	24.75	32.26	37.86	35.15	38.06	34.99	37.96
	A-MEM	34.83	48.41	12.46	34.65	41.53	25.00	40.28	29.10	39.04	27.10
	Mem0	11.11	30.10	26.88	43.07	36.59	38.57	33.64	41.04	33.16	39.81
	LangMem	24.04	27.07	16.37	20.00	23.54	37.14	34.42	31.34	37.17	34.24
	MemoryOS	38.55	44.59	13.26	36.00	40.30	19.29	42.43	18.66	42.95	18.98
	MemSkill	**39.28**	**52.07**	23.75	**59.90**	**55.99**	**60.00**	**24.54**	**64.18**	**23.57**	**62.09**

Bold indicates the best score within each base model block.

▲ indicates no training using this base model or dataset (transfer evaluation only).

实现细节。我们将控制器初始化为一个轻量级的多层感知机（MLP），并使用 LLaMA-3.3-70B-Instruct (Grattafiori et al., 2024) 和 Qwen3-Next-80B-A3B-Instruct (Yang et al., 2025) 作为基础大模型，通过 API 服务进行访问。除非另有说明，我们仅在 LLaMA 上训练 MemSkill，而仅在迁移实验中使用 Qwen。LongMemEval 也在迁移情景下进行评估，其中我们直接应用在 LoCoMo 上学成的技能，无需进一步训练。

对于 MemSkill 和所有基准方法，我们为保持一致的对比，最多检索 20 个记忆项。在训练过程中，我们使用 PPO (Schulman et al., 2017) 初始化控制器最优化。MemSkill 在跨度级别执行记忆构建。在对话基准测试中，我们将每个对话会话作为训练期间的基本处理单元，控制器在每个单元中选择少量技能，使用 K=3 。我们采用 Qwen3-Embedding-0.6B (Yang et al., 2025) 作为状态和技能表示的共享编码器，并将 Contriever (Izacard et al., 2021) 作为默认的记忆检索器。对于设计者，我们每 100 个训练步骤触发一次技能演化，并且每轮演化最多允许 3 次技能编辑。对于 ALFWorld，我们将最大环境交互长度限制为 50 步。

在评估时，我们保持相同的跨度级公式，将跨度/块大小默认设置为 512，同时保持整体流程不变。除非另有说明，我们在评估时对 LoCoMo 和 LongMemEval 使用 K=7，对 ALFWorld 使用 K=5。更多的实现细节和提示模板见附录 A和附录 C。

4.2. 对比实验

在对话与具身情景中的有效性。表 1 总结了在 LoCoMo、LongMemEval 和 ALFWorld 上的主要对比结果。在这些数据集上，MemSkill 在所有对比方法中表现出最强的总体性能。在对话基准测试中，MemSkill 在每个基础模型块内于 LoCoMo 和 LongMemEval 上均取得了最优的 LLM 评分，表明其构建的记忆质量更高。相比之下，以往的方法如 MemoryBank、A-MEM 和 MemoryOS 使用固定且手动指定的记忆提取与修订过程，而 MemSkill 则通过交互学习并演化其技能，从而在不同情境下具备更强的适应能力。在 ALFWorld 上，MemSkill 在已见和未见划分上均取得了最高的成功率，表明基于技能的记忆构建有助于交互式决策，而其他基线方法在利用记忆支持长时程动作执行方面表现较差。总体而言，结果表明 MemSkill 在多种情景下均具有良好的有效性。

基础模型间的泛化能力。MemSkill 的一个关键优势在于其在不同基础模型间具有强大的泛化能力。我们仅使用 LLaMA 训练 MemSkill，并直接将学成的技能迁移到 Qwen 上，无需重新训练。尽管这种迁移情景非常严格，MemSkill 依然保持高度竞争力，且在对话和具身评估中持续优于多个强基准，这表明所演化的技能捕捉到了可复用的记忆行为，能够被不同潜在的大模型实例化。

跨数据集迁移。 MemSkill 在同一广泛情景内的不同数据集之间也表现出良好的泛化能力。具体而言，LongMemEval 仅通过将从 LoCoMo 学成的技能库进行迁移来评估，但 MemSkill 仍取得了所有方法中的最佳结果，表明学成的技能并未过度拟合于单一基准。我们在第 4.4 节中进一步研究了在更显著分布偏移下的迁移表现。

4.3. 消融研究

我们进行消融实验，以分离（i）学习选择技能和（ii）演化技能库的贡献。表 2 报告了在基础模型（LLaMA 和 Qwen）下，LoCoMo 上的 LLM 判官（L-J）结果。如表所示，**无控制器（随机技能）**用随机技能选择替代学成的控制器，其余流水线保持不变。**无设计者（静态技能）**关闭设计者，将技能库固定为最初的四个基本技能。**仅精炼（不添加新技能）**允许设计者对现有技能进行精炼，但禁止添加新技能。

在两个基础模型中，移除任一组件都会持续降低性能，证实了 MemSkill 同时受益于目标技能选择和技能演化。特别是，随机选择技能会导致性能明显下降，突显出学习选择相关技能的重要性，而非随意提供技能。关闭设计者导致的性能下降更为严重，尤其是在 Qwen 上表现尤为明显，表明演化技能库对于学习可复用的记忆行为至关重要，这些行为能够泛化到固定的手动指定操作集之外。最后，仅进行精炼的方法在 LLaMA 和 Qwen 上均优于静态技能，尤其在 Qwen 上提升显著，但仍低于默认设置，表明引入新技能所带来的收益超出了对初始原始技能的精炼。

Table 2. **基于 L-J 指标的 LoCoMo 消融研究**

Variant	LLaMA	Qwen
MemSkill (default)	**50.96**	**52.07**
w/o controller (random skills)	45.86	41.24
w/o designer (static skills)	44.11	34.71
Refine-only (no new skills)	44.90	46.97

4.4. 分布偏移下的技能泛化

除了在对话式记忆基准上的迁移之外，我们还评估了学成技能在交互格式和证据结构分布发生变化时的泛化能力。具体而言，我们将基于 LoCoMo 训练的技能库直接应用于 HotpotQA，其中输入为长篇文档式叙述，而非多轮对话。遵循 (Yu et al., 2025) 中的评估协议，我们测试了三种上下文长度设置，难度逐级增加，对应于不同数量的连结文档（即

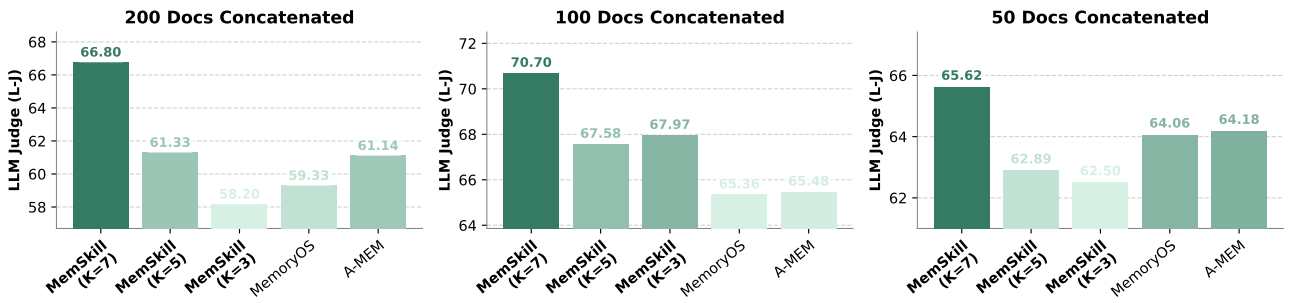

Figure 3. **HotpotQA 上分布偏移下的技能泛化能力**。我们将 LoCoMo 训练的技能库迁移至 HotpotQA，并按照 (Yu et al., 2025) 的方法评估三种上下文长度设置（50/100/200 个连结文档）。柱状图展示了在不同 Top-K 技能数量下，基于 LLaMA 的 LLM 评判器（L-J）的表现，与 MemoryOS 和 A-MEM 进行对比。

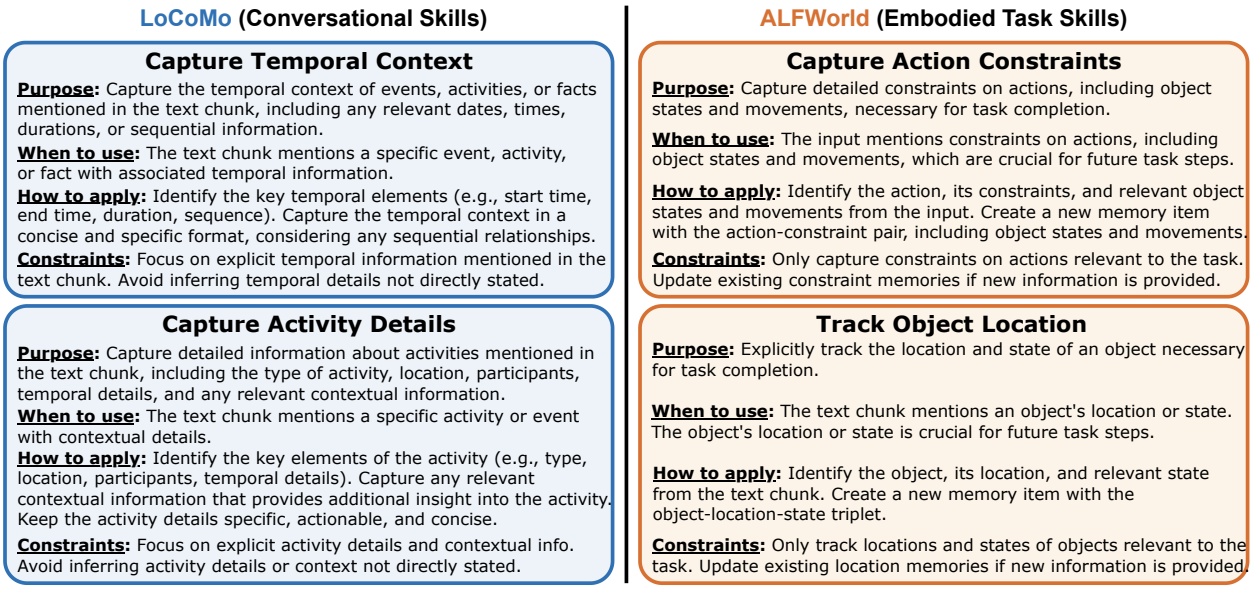

Figure 4. **案例研究**。我们展示了在 LoCoMo 和 ALFWorld 上学成的代表性技能。（“描述”因简洁性而省略。）

50/100/200）。本节所有结果均使用 LLaMA 作为基础模型，并报告大语言模型评分器得分（L-J）。对于基准方法，我们包含 MemoryOS 和 A-MEM，它们是表 1中对话基准上最具竞争力的方法，为清晰起见，省略了较弱的替代方案。

图 3 表明，MemSkill 在所有三种上下文大小下均能有效迁移至 HotpotQA。特别是，在更具挑战性的长上下文情景中，MemSkill 一直优于 MemoryOS 和 A-MEM 等强基准模型，且优势更加显著。这些结果表明，学成的记忆技能并非局限于对话特定的表面形式，而是捕捉到了可复用的提取与修订行为，即使输入结构和检索需求发生变化，这些行为依然有效。

相同的图表也揭示了对所选技能数量的轻微敏感性 K。增加 K 通常会提升性能，其中 K=7 在所有三种情景下均取得了最佳结果，而较小的 K 在较长上下文下可能无法充分利用技能库。总体而言，这一趋势表明，当上下文变得更长且更嘈杂时，MemSkill 通过组合多个技能受益，同时在无需任何 HotpotQA 特定训练的情况下仍保持强大的迁移能力。

4.5. 案例研究

为了使 MemSkill 更具可解释性，我们检查了最终演化出的技能库，并报告了从 LoCoMo 和 ALFWorld

中学成的代表性技能。如图 4 所示，学成的技能在 LoCoMo 和 ALFWorld 之间表现出明显的领域特化。对于 LoCoMo，图 4 中的技能强调时间上下文和活动细节，表明有效的对话记忆通常得益于以轻量级结构组织事件，例如谁在何时何地做了什么，从而支持长时间交互中的信息管理。更广泛地说，演化出的技能库反映了数据中反复出现的信息需求，而非对应该记住什么的单一固定概念。相比之下，ALFWorld 的技能则聚焦于动作约束和物体位置，突显出具身成功依赖于维护一个可操作的世界状态摘要，包括任务相关的先决条件，而非宽泛的叙事总结，以支持多步骤执行。

综上所述，这些技能展示了 MemSkill 如何从交互数据中自动提炼出可重复使用的记忆行为，并通过训练不断优化它们，从而朝着更适应性的记忆系统发展，同时减少对人工设计的记忆结构的依赖。更多演化的技能详见附录 B。

5. 结论

我们提出 **MemSkill**，一种将记忆操作重构为动态演化的技能库的智能体记忆方法。MemSkill 学习为每个上下文片段选择一组相关的少量技能，并基于这些技能对大型语言模型执行器进行条件化，以实现受技能引导的记忆构建。除了学习如何使用固定的运算集合外，MemSkill 还引入了一个设计者，通过优化现有技能并从具有挑战性的案例中提出新技能来改进技能库本身，从而形成一个闭环训练流程。在 LoCoMo、LongMemEval、HotpotQA 和 ALFWorld 上的实验表明，该方法在多个强基准上均实现了持续提升；定性分析也展示了演进的技能如何带来更具适应性的记忆管理行为。我们希望 MemSkill 能够激励未来对自提升智能体记忆系统的研究，使智能体不仅学会使用记忆，还能持续改进记忆的构建与维护方式。

致谢

本研究/项目得到了新加坡南洋理工大学启动基金（#023284-00001）和新加坡教育部科研基金一级种子基金（RS37/24，#025041-00001）的支持。

影响声明

MemSkill 通过将注意力从静态的手工设计流程转向可学习且可演化的记忆技能，推动了智能体记忆设计的进展。这一视角使长时间运行的大语言模型智能体在交互历史不断增长、关键信息随时间变化的情景中更具实用性。通过改进记忆的提取、整合与修订方式，MemSkill 能够支持更一致的服务，适用于多会话个人助手、教育辅导、长篇客户支持以及互动研究工具等场景，在这些场景中，智能体需在保留相关上下文的同时避免冗余或过时信息。

除了即时应用之外，MemSkill 还提供了一种可重复使用的方法论，用于研究记忆行为的定义与优化方式。明确的技能库为检查和分析提供了具体的接口，可能促进更具可解释性和可控性的记忆系统的发展。更广泛地说，从棘手案例中迭代改进记忆管理行为的理念，可以启发其他智能体子系统（如工具使用或规划）采用类似的自我改进机制，这些领域目前仍普遍依赖固定的启发式方法。

与任何具备记忆功能的智能体一样，负责任地使用需要基本的安全保障。例如，部署时应避免存储不必要的敏感信息，并应提供面向用户的功能，用于检查和删除记忆内容。这些考虑因素是增强记忆系统中的常规做法，并非 MemSkill 所独有，但随着智能体记忆功能越来越高效并被广泛采用，其重要性也日益凸显。

参考文献

Chen, Y., Wang, Y., Zhu, S., Yu, H., Feng, T., Zhang, M., Patwary, M., and You, J. Multi-agent evolve: Llm self-improve through co-evolution. *arXiv preprint arXiv:2510.23595*, 2025.

Chhikara, P., Khant, D., Aryan, S., Singh, T., and Yadav, D. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

Fang, J., Deng, X., Xu, H., Jiang, Z., Tang, Y., Xu, Z., Deng, S., Yao, Y., Wang, M., Qiao, S., et al. Lightmem: Lightweight and efficient memory-augmented generation. *arXiv preprint arXiv:2510.18866*, 2025.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Hu, S., Lu, C., and Clune, J. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.

Hu, Y., Liu, S., Yue, Y., Zhang, G., Liu, B., Zhu, F., Lin, J., Guo, H., Dou, S., Xi, Z., et al. Memory in the age of ai agents. *arXiv preprint arXiv:2512.13564*, 2025.

Huang, C., Yu, W., Wang, X., Zhang, H., Li, Z., Li, R., Huang, J., Mi, H., and Yu, D. R-zero: Self-evolving reasoning llm from zero data. *arXiv preprint arXiv:2508.05004*, 2025.

Izacard, G., Caron, M., Hosseini, L., Riedel, S., Bojanowski, P., Joulin, A., and Grave, E. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118*, 2021.

Kang, J., Ji, M., Zhao, Z., and Bai, T. Memory os of ai agent. *arXiv preprint arXiv:2506.06326*, 2025.

Kool, W., Van Hoof, H., and Welling, M. Stochastic beams and where to find them: The gumbel-top-k trick for sampling sequences without replacement. In *International conference on machine learning*, pp. 3499–3508. PMLR, 2019.

LangChain. Langmem. https://github.com/langchain-ai/langmem, 2025. GitHub repository.

Lee, K.-H., Chen, X., Furuta, H., Canny, J., and Fischer, I. A human-inspired reading agent with gist memory of very long contexts. *arXiv preprint arXiv:2402.09727*, 2024.

Maharana, A., Lee, D.-H., Tulyakov, S., Bansal, M., Barbieri, F., and Fang, Y. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753*, 2024.

Novikov, A., Vũ, N., Eisenberger, M., Dupont, E., Huang, P.-S., Wagner, A. Z., Shirobokov, S., Kozlovskii, B., Ruiz, F. J., Mehrabian, A., et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

Packer, C., Fang, V., Patil, S., Lin, K., Wooders, S., and Gonzalez, J. Memgpt: Towards llms as operating systems. 2023.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Shridhar, M., Yuan, X., Côté, M.-A., Bisk, Y., Trischler, A., and Hausknecht, M. Alfworld: Aligning text and embodied environments for interactive learning. *arXiv preprint arXiv:2010.03768*, 2020.

Wang, Y., Takanobu, R., Liang, Z., Mao, Y., Hu, Y., McAuley, J., and Wu, X. Mem-{\alpha}: Learning memory construction via reinforcement learning. *arXiv preprint arXiv:2509.25911*, 2025a.

Wang, Z., Wang, K., Wang, Q., Zhang, P., Li, L., Yang, Z., Jin, X., Yu, K., Nguyen, M. N., Liu, L., et al. Ragen: Understanding self-evolution in llm agents via multi-turn reinforcement learning. *arXiv preprint arXiv:2504.20073*, 2025b.

Wei, T., Sachdeva, N., Coleman, B., He, Z., Bei, Y., Ning, X., Ai, M., Li, Y., He, J., Chi, E. H., et al. Evo-memory: Benchmarking llm agent test-time learning with self-evolving memory. *arXiv preprint arXiv:2511.20857*, 2025.

Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.-W., and Yu, D. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.

Wu, R., Wang, X., Mei, J., Cai, P., Fu, D., Yang, C., Wen, L., Yang, X., Shen, Y., Wang, Y., et al. Evolver: Self-evolving llm agents through an experience-driven lifecycle. *arXiv preprint arXiv:2510.16079*, 2025.

Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., and Zhang, Y. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.

Yan, S., Yang, X., Huang, Z., Nie, E., Ding, Z., Li, Z., Ma, X., Kersting, K., Pan, J. Z., Schütze, H., et al. Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*, 2025.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W., Salakhutdinov, R., and Manning, C. D. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pp. 2369–2380, 2018.

Yu, H., Chen, T., Feng, J., Chen, J., Dai, W., Yu, Q., Zhang, Y.-Q., Ma, W.-Y., Liu, J., Wang, M., et al. Memagent: Reshaping long-context llm with multi-conv rl-based memory agent. *arXiv preprint arXiv:2507.02259*, 2025.

Yu, W., Zhang, H., Pan, X., Cao, P., Ma, K., Li, J., Wang, H., and Yu, D. Chain-of-note: Enhancing robustness in retrieval-augmented language models. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pp. 14672–14685, 2024.

Zhai, Y., Tao, S., Chen, C., Zou, A., Chen, Z., Fu, Q., Mai, S., Yu, L., Deng, J., Cao, Z., et al. Agentevolver: Towards efficient self-evolving agent system. *arXiv preprint arXiv:2511.10395*, 2025.

Zhang, G., Ren, H., Zhan, C., Zhou, Z., Wang, J., Zhu, H., Zhou, W., and Yan, S. Memevolve: Meta-evolution of agent memory systems. *arXiv preprint arXiv:2512.18746*, 2025.

Zhao, A., Huang, D., Xu, Q., Lin, M., Liu, Y.-J., and Huang, G. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19632–19642, 2024.

Zhao, A., Wu, Y., Yue, Y., Wu, T., Xu, Q., Lin, M., Wang, S., Wu, Q., Zheng, Z., and Huang, G. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025.

Zheng, B., Fatemi, M. Y., Jin, X., Wang, Z. Z., Gandhi, A., Song, Y., Gu, Y., Srinivasa, J., Liu, G., Neubig, G., et al. Skillweaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025.

Zhong, W., Guo, L., Gao, Q., Ye, H., and Wang, Y. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19724–19731, 2024.

A. 更多实施细节

A.1. 评估细节

大模型评判器与基础设施。我们使用 `openai/gpt-oss-120b` 作为大模型评判器。所有基于 API 的模型均通过 NV NIM API 和 Together API 访问。训练在 NVIDIA A6000 GPU 上进行。

LoCoMo (Maharana et al., 2024). LoCoMo 包含 10 个长交互样本，每个样本平均配以约 200 个训练查询。我们按样本将数据集划分为训练/验证/测试集，比例为 6:2:2。此外，我们还移除了 *对抗*查询，因为它们的证据不在提供的上下文中，可能在训练过程中引入噪声监督。

LongMemEval (Wu et al., 2024). 我们使用 LongMemEval-S 数据集划分，其中每个样本包含约 100K token 的超长对话。随后，我们在数据集的一个分层样本上进行迁移评估，该样本约为数据集的五分之一（约 100 个样本），以确保覆盖不同类型的题目，从而进行全面评估。

ALFWorld (Shridhar et al., 2020). 我们首先从训练集收集专家轨迹，并将其作为记忆或经验构建的语料库。然后在官方的 ALF-Seen 和 ALF-Unseen 测试集上进行评估。

HotpotQA (Yang et al., 2018). 我们使用 HotpotQA 来研究分布偏移下的迁移性能，遵循 (Yu et al., 2025) 的评估协议。具体而言，我们在三种上下文长度设置下进行评估，难度逐步增加，分别对应 50/100/200 个连结文档（记为 `eval_50`、`eval_100` 和 `eval_200`）。除非另有说明，本部分所有结果均以 LLaMA 作为基础模型，并报告 LLM 判分（L-J）。

跨度级别评估。在评估过程中，我们以默认跨度大小为 512 个 token 的方式在跨度级别上进行记忆构建，而不是逐轮更新记忆。这显著减少了大语言模型调用次数，提升了评估效率。

A.2. 设计师的更多详情

硬壳缓冲与代表性案例挖掘。 设计者维护一个滑动的*硬案例缓冲区*，用于跟踪最近具有挑战性的评估案例，且不会无限增长。每个案例存储查询、用于回答该查询的检索记忆、模型预测、参考答案、任务奖励得分（例如 F1），以及一个失败计数器，记录该案例被错误回答的次数。为了优先处理那些奖励得分低且反复失败的案例，我们为每个案例分配一个难度得分

$$d(q) \;=\; \big(1 - r(q)\big) \cdot c(q), \tag{5}$$

其中 $r(q) \in [0, 1]$ 是查询 q 的任务奖励，$c(q)$ 是其在缓冲窗口内的累计失败次数。较高的 $d(q)$ 表示更关键的情况，应优先检查。

为了鼓励对*多样化*错误类型的覆盖，我们进一步根据查询的语义相似度对困难案例进行聚类，并从每个簇中挖掘代表性案例。例如，在 LoCoMo 中，某些查询关注时间线索（如事件发生的*何时*），而另一些则强调位置信息（如某事发生的*何处*）。聚类有助于分离这些语义类型，避免设计者反馈被单一频繁的错误模式所主导，从而提升挖掘出的监督信号的多样性和完整性。

新引入技能的探索激励。 每次演化回合后，设计师可能会引入控制器尚未学成的新技能以供利用。为促进采纳，我们在更新后采用一个简短的探索阶段，直接在 Logit 层面将控制器偏向于新技能。令 $\mathcal{S}_{\text{new}} \subseteq \mathcal{S}$ 表示新增技能的集合，令 $p_\theta(i \mid s_t) = \text{softmax}(z_t)_i$ 为步骤 t 时控制器的分布。我们要求分配给新技能的总概

率质量至少达到目标阈值 τ_t：

$$\sum_{i \in \mathcal{S}_{\text{new}}} p_\theta(i \mid s_t) \;\geq\; \tau_t, \qquad \tau_t \in [0,1]. \tag{6}$$

当公式 (6) 中的约束被违反时，我们向所有新技能添加一个均匀的 Logit 增益 δ_t，

$$z'_{t,i} \;=\; \begin{cases} z_{t,i} + \delta_t, & i \in \mathcal{S}_{\text{new}}, \\ z_{t,i}, & \text{otherwise}, \end{cases} \qquad p'_\theta(\cdot \mid s_t) = \text{softmax}(z'_t), \tag{7}$$

其中，δ_t 被选为使 $\sum_{i \in \mathcal{S}_{\text{new}}} p'_\theta(i \mid s_t) \geq \tau_t$ 成立的最小值。通过作用于 Logit，该机制保持了控制器架构，并在概率层面平滑地鼓励新技能。

我们在每次演化回合后，对前 T_{explore}=50 个训练步骤应用此激励。为避免持续的偏差，目标阈值在此期间线性衰减：

$$\tau_t \;=\; \tau_0 \cdot \Big(1 - \frac{t}{T_{\text{explore}}}\Big), \qquad t = 0, 1, \ldots, T_{\text{explore}}, \tag{8}$$

默认 τ_0=0.3 。该调度策略提供了强劲的初始探索，随后逐渐减弱，从而实现平滑地过渡回控制器的学成选择行为。

基于稳定奖励的早停与回滚。 MemSkill 每隔一段时间执行一次技能演化，每次演化周期包含在当前技能库上固定数量的控制器训练步骤（例如 100 步）。由于技能库更新后奖励信号可能具有波动性，我们通过使用一个 稳定化的奖励估计来评估周期是否提升了性能：计算该周期内最后四分之一训练步骤中的平均任务奖励，并将此值视为该周期的得分。

令 L 表示每个周期内的控制器训练步数，$\{r_t\}_{t=1}^{L}$ 表示周期内的步级奖励。我们将周期得分定义为

$$\bar{r}_{\text{tail}} \;=\; \frac{1}{L/4} \sum_{t=3L/4+1}^{L} r_t. \tag{9}$$

我们将 $\bar{r}_{\text{tail}}$ 与迄今为止观察到的最佳得分进行比较。如果当前周期未能改善该准则，则在执行下一步技能演化之前，我们将技能库回滚到之前表现最佳的快照，并从该快照重新开始演化。这种回滚机制可防止次优设计者更新导致的性能退化累积。

最后，如果经过若干连续的演化周期后，稳定后的奖励仍未改善（我们使用固定的耐心值），则提前停止训练，并返回训练过程中遇到的最佳技能库快照。

A.3. ALFWorld 训练详情

ALFWorld 与其他基准的不同之处在于，它是一个交互式环境，而非静态文本语料库。为了在此情景下实例化 MemSkill，我们首先通过在训练集上收集专家轨迹，将 ALFWorld 转换为离线训练协议。每条轨迹记录了智能体的交互序列（观测、动作和结果），并作为记忆构建的交互迹。

任务型分组。 ALFWorld 任务自然地归入少数几种重复出现的目标模板。按照常规做法，我们按任务类型（即目标模板）对轨迹进行分组，例如 Pick & Place（将一个物体放入/放到目标容器中）、Clean & Place（清洁一个物体后再放置）、Heat & Place（加热一个物体后再放置），以及 Cool & Place（冷却一个物体后再放置）。[1]

[1]我们使用环境提供的任务模板来定义任务类型。

经验语料库与评估案例。 为了将 ALFWorld 适配到我们的训练框架中，我们从离线专家轨迹中构建每类任务的训练时数据划分。对于每种任务类型，我们随机采样一部分轨迹作为用于记忆构建的 *经验语料库*，并从同一类型中再采样另一组 *不重叠*的轨迹作为 *评估样本*。在训练过程中，MemSkill 从经验语料库中（逐跨度地，通过控制器和执行器）构建特定轨迹的记忆库，然后在评估样本上评估所构建的记忆，以获得任务奖励并记录失败案例。

动机。 使用来自*相同*任务类型的非重叠轨迹进行经验构建和评估，提供了受控的泛化信号：同一类型内的轨迹共享目标结构和重复的交互模式，使得记忆和技能在相同模板的不同实例间更具可迁移性。这种设置促使 MemSkill 学习能够捕捉类型级别规律（例如相关对象状态和动作先决条件）的可复用记忆技能，而非过度拟合到单个轨迹，同时仍确保评估迹线与用于构建记忆的迹线相互独立。

A.4. 训练目标详情

本部分详细介绍了在 MemSkill 中，当每次决策选择一个不重复的有序前 K 集技能时，用于优化控制器的强化学习目标。

回合、状态和前-K 个动作。 训练迭代遍历交互迹（回合）。对于一条迹，MemSkill 依次处理文本片段。在步骤 t ，控制器观察到一个状态 $s_t \triangleq (x_t, M_t)$ ，该状态由当前文本片段 x_t 和从迹特定记忆库中检索到的记忆 M_t 组成。令 $\mathcal{S}_t = \{1, \ldots, N_t\}$ 表示当前技能库，其大小 N_t 可能随设计者演化技能而变化。控制器输出 Logit $z_\theta(s_t) \in \mathbb{R}^{N_t}$ ，并诱导出一个类别分布

$$p_\theta(i \mid s_t) = \mathrm{softmax}(z_\theta(s_t))_i. \tag{10}$$

与其采样单一技能，控制器会选择一个 *有序*的前 K 项集合 $A_t = (a_{t,1}, \ldots, a_{t,K})$ *不重复地*，通过 Gumbel-Top-K 采样实现 (Kool et al., 2019)(i.e., adding i.i.d. Gumbel noise to logits and taking the top-K indices)。

Top-K 无放回选取的联合概率。 对于 PPO 风格的策略最优化，我们需要在不放回采样过程中采样有序集合 A_t 的联合概率。该概率可表示为

$$\pi_\theta(A_t \mid s_t) = \prod_{j=1}^{K} \frac{p_\theta(a_{t,j} \mid s_t)}{1 - \sum_{\ell<j} p_\theta(a_{t,\ell} \mid s_t)}, \tag{11}$$

相应的联合对数概率

$$\log \pi_\theta(A_t \mid s_t) = \sum_{j=1}^{K} \Big(\log p_\theta(a_{t,j} \mid s_t) - \log \big(1 - \sum_{\ell<j} p_\theta(a_{t,\ell} \mid s_t)\big) \Big). \tag{12}$$

当 $K = 1$ 时，式 11 退化为标准的单动作情形。

基于记忆的评估所获得的奖励。 对于每个迹，处理完所有跨度并构建出与迹相关的记忆库后，我们将在该迹的记忆依赖型训练查询上评估记忆库，并获得一个标量任务得分（例如 F1 值或成功概率）。我们将此得分视为回合级别的奖励：

$$R \triangleq \mathrm{Eval}(\text{memory bank}; \text{training queries}) \in \mathbb{R}. \tag{13}$$

该奖励随后被分配给轨迹中的一系列控制器决策。具体而言，我们使用带有折扣因子 γ 的标准总回报计算方法。

$$G_t = \sum_{\tau=t}^{T} \gamma^{\tau-t} r_\tau, \tag{14}$$

其中 r_τ 是每步的奖励。在我们的默认情景中，仅在记忆构建完成后提供奖励，即 $r_T = R$ 和 $r_\tau = 0$ 对应 $\tau < T$ ，因此 $G_t = \gamma^{T-t} R$ 。我们学习一个价值函数 $V_\phi(s_t)$ ，并使用泛化优势估计（GAE）计算优势 $\hat{A}_t$ 。

PPO 目标函数，使用前-K 个动作。 我们使用近端策略优化（PPO）(Schulman et al., 2017) 对控制器进行最优化，将标准的单动作概率替换为公式 12 中的前 K 个动作的联合概率。令 θ_{old} 表示用于收集轨迹的行为策略的参数。定义重要性比率

$$r_t(\theta) = \frac{\pi_\theta(A_t \mid s_t)}{\pi_{\theta_{\text{old}}}(A_t \mid s_t)} = \exp\Big(\log \pi_\theta(A_t \mid s_t) - \log \pi_{\theta_{\text{old}}}(A_t \mid s_t)\Big). \tag{15}$$

截断的替代策略目标是

$$\mathcal{L}_{\text{policy}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\,\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\big)\Big]. \tag{16}$$

我们还优化了一个价值函数，并加入了熵奖励以促进探索：

$$\mathcal{L}_{\text{value}}(\phi) = \mathbb{E}_t\Big[\big(V_\phi(s_t) - G_t\big)^2\Big], \tag{17}$$

$$\mathcal{H}(\theta) = \mathbb{E}_t\big[H(p_\theta(\cdot \mid s_t))\big], \tag{18}$$

其中 $H(\cdot)$ 是所有技能上的类别分布的熵。整体目标（需最大化）为

$$\max_{\theta,\phi}\ \mathcal{L}_{\text{policy}}(\theta) - c_v\,\mathcal{L}_{\text{value}}(\phi) + c_H\,\mathcal{H}(\theta). \tag{19}$$

在实现中，我们最小化公式 19 的负值。

Gumbel-Top-K 探索。 在回溯采样过程中，为无放回地采样前 K 个技能，我们采用 Gumbel-Top-K 采样：在每一步中，我们独立同分布地抽取 Gumbel 噪声 $\{g_i\}_{i=1}^{N_t}$ ，构造扰动后的 Logit $\tilde{z}_i = z_i + g_i$ ，并选取 K 个最大的 $\tilde{z}_i$ 对应的索引以获得 A_t 。这在保持与 PPO 兼容性的同时，通过公式 11 中的联合概率实现了对技能子集的随机探索。为了训练稳定性，熵正则化是从所有技能上的基础类别分布 $p_\theta(\cdot \mid s_t)$ 中计算得到的（公式 18），即使实际执行的动作是前 K 个技能的集合，该正则化仍能鼓励对不断演化的技能库进行探索。

B. 案例研究

B.1. 初始原始技能

Initial Primitive Skill - INSERT

Skill: Insert New Memory

Description: Memory management skill for capturing new, durable facts from the current text chunk that are not already in memory.

Purpose: Capture new, durable facts from the current text chunk that are missing in memory.

When to use:
- The text chunk introduces new facts, events, plans, or context worth storing.
- The information is stable and likely useful later.

How to apply:
- Compare against retrieved memories to avoid duplicates.
- Split distinct facts into separate items.
- Keep each item concise and specific.

Constraints:
- Skip trivial, fleeting, or speculative content.
- Do not update or delete existing memories.

Action type: INSERT only.

Initial Primitive Skill - UPDATE

Skill: Update Existing Memory

Description: Memory management skill for revising an existing memory item when the text chunk provides corrections or new details.

Purpose: Revise a retrieved memory with new or corrected information from the text chunk.

When to use:
- The text chunk clarifies, corrects, or extends a retrieved memory.

How to apply:
- Select the best matching memory item.
- Merge new details into a single updated item.
- Preserve accurate details that still hold.

Constraints:
- Do not create new memories.
- Do not delete items.

Action type: UPDATE only.

Initial Primitive Skill - DELETE

Skill: Delete Invalid Memory

Description: Memory management skill for removing memory items that are incorrect, outdated, or superseded.

Purpose: Remove a retrieved memory that is wrong, outdated, or superseded by the text chunk.

When to use:
- The text chunk clearly contradicts a memory.
- A plan or fact is explicitly canceled or replaced.

How to apply:
- Only delete when evidence is explicit.

Constraints:
- If uncertain, prefer no action over deletion.

Action type: DELETE only.

Initial Primitive Skill - SKIP

Skill: No Operation

Description: Memory management skill for confirming that no memory changes are required.

Purpose: Confirm no memory changes are needed for the text chunk.

When to use:
- The text chunk contains no new, corrective, or actionable information.

Constraints:
- Emit NOOP only if none of the selected skills produce actions.

Action type: NOOP only.

B.2. LoCoMo 上的进化技能

Evolved Skill on LoCoMo - CAPTURE_ACTIVITY_DETAILS

Skill: Capture Activity Details

Purpose: Capture detailed information about activities mentioned in the text chunk, including the type of activity, location, participants, temporal details, and any relevant contextual information.

When to use:
- The text chunk mentions a specific activity or event with contextual details.

How to apply:
- Identify the key elements of the activity (e.g., type, location, participants, temporal details).
- Capture any relevant contextual information that provides additional insight into the activity.
- Keep the activity details specific, actionable, and concise.

Constraints:
- Focus on explicit activity details and contextual information mentioned in the text chunk.
- Avoid inferring activity details or context not directly stated.

Action type: INSERT only.

Evolved Skill on LoCoMo - CAPTURE_ENTITY_NUANCES

Skill: Capture Entity Nuances

Purpose: Capture nuanced details about entities mentioned in the text chunk, such as nicknames, aliases, or comparative statements.

When to use:
- The text chunk mentions an entity with nuanced details (e.g., nickname, alias, comparison).

How to apply:
- Identify the entity and its associated nuances.
- Capture these nuances in a way that distinguishes them from the entity's primary information.

Constraints:
- Focus on explicit nuances mentioned in the text chunk.
- Avoid inferring nuances not directly stated.

Action type: INSERT only.

Evolved Skill on LoCoMo - CAPTURE_TEMPORAL_CONTEXT

Skill: Capture Temporal Context

Purpose: Capture the temporal context of events, activities, or facts mentioned in the text chunk, including any relevant dates, times, durations, or sequential information.

When to use:
- The text chunk mentions a specific event, activity, or fact with associated temporal information.

How to apply:
- Identify the key temporal elements (e.g., start time, end time, duration, sequence).
- Capture the temporal context in a concise and specific format, considering any sequential relationships.

Constraints:
- Focus on explicit temporal information mentioned in the text chunk.
- Avoid inferring temporal details not directly stated.

Action type: INSERT only.

Evolved Skill on LoCoMo - DELETE

Skill: Delete Invalid Memory

Purpose: Remove a retrieved memory that is wrong, outdated, or superseded by the text chunk.

When to use:
- The text chunk clearly contradicts a memory.
- A plan or fact is explicitly canceled or replaced.

How to apply:
- Only delete when evidence is explicit.

Constraints:
- If uncertain, prefer no action over deletion.

Action type: DELETE only.

Evolved Skill on LoCoMo - HANDLE_ENTITY_RELATIONSHIPS

Skill: Handle Entity Relationships

Purpose: Capture and manage complex relationships between entities mentioned in the text chunk, including nuanced details.

When to use:
- The text chunk mentions interactions, associations, or relationships between entities with specific details.

How to apply:
- Identify the entities involved and their roles in the relationship.
- Capture the nature of the relationship and any nuanced details (e.g., nicknames, comparative statements).
- Update existing memories to reflect the new relationship information.

Constraints:
- Focus on explicit relationships mentioned in the text chunk.
- Avoid inferring relationships not directly stated.

Action type: INSERT only.

Evolved Skill on LoCoMo - INSERT

Skill: Insert New Memory

Purpose: Capture new, durable facts from the current text chunk that are missing in memory, including specific temporal details such as dates or time frames and detailed activity information.

When to use:
- The text chunk introduces new facts, events, plans, or context worth storing.
- The information is stable and likely useful later.

How to apply:
- Compare against retrieved memories to avoid duplicates.
- Split distinct facts into separate items.
- Keep each item concise and specific, including relevant temporal information and activity details.

Constraints:
- Skip trivial, fleeting, or speculative content.
- Do not update or delete existing memories.

Action type: INSERT only.

Evolved Skill on LoCoMo - NOOP

Skill: No Operation

Purpose: Confirm no memory changes are needed for the text chunk.

When to use:
- The text chunk contains no new, corrective, or actionable information.

Constraints:
- Emit NOOP only if none of the selected skills produce actions.

Action type: NOOP only.

Evolved Skill on LoCoMo - REFINE_TEMPORAL_DETAILS_WITH_CONTEXT

Skill: Refine Temporal Details with Context

Purpose: Update the temporal context of existing memories with new information from the text chunk, considering the context in which the information is provided.

When to use:
- The text chunk provides new or corrected temporal information relevant to an existing memory, and the context suggests a need for refinement.

How to apply:
- Identify the relevant existing memory and its current temporal context.
- Update the temporal details to reflect the new information, ensuring consistency with the provided context.

Constraints:
- Focus on explicit temporal information mentioned in the text chunk and supported by the context.
- Avoid inferring temporal details not directly stated or implied by the context.

Action type: UPDATE only.

Evolved Skill on LoCoMo - UPDATE

Skill: Update Existing Memory

Purpose: Revise a retrieved memory with new or corrected information from the text chunk, including entity-specific details.

When to use:
- The text chunk clarifies, corrects, or extends a retrieved memory.
- The text chunk provides new information about a specific entity or its activities.

How to apply:
- Select the best matching memory item.
- Merge new details into a single updated item.
- Preserve accurate details that still hold, and ensure entity-specific information is accurately captured and updated.

Constraints:
- Do not create new memories.
- Do not delete items.

Action type: UPDATE only.

B.3. ALFWorld 中的进化技能

Evolved Skill on ALFWorld - CAPTURE_ACTION_CONSTRAINTS

Skill: Capture Action Constraints

Purpose: Capture detailed constraints on actions, including object states and movements, necessary for task completion.

When to use:
- The text chunk mentions constraints on actions, including object states and movements.
- The constraints are crucial for future task steps.

How to apply:
- Identify the action, its constraints, and relevant object states and movements from the text chunk.
- Create a new memory item with the action-constraint pair, including object states and movements.

Constraints:
- Only capture constraints on actions relevant to the task.
- Update existing constraint memories if new information is provided.

Action type: INSERT only.

Evolved Skill on ALFWorld - DELETE

Skill: Delete Invalid Memory

Purpose: Remove a retrieved memory that is wrong, outdated, or superseded by the text chunk.

When to use:
- The text chunk clearly contradicts a memory.
- A plan or fact is explicitly canceled or replaced.

How to apply:
- Only delete when evidence is explicit.

Constraints:
- If uncertain, prefer no action over deletion.

Action type: DELETE only.

Evolved Skill on ALFWorld - INSERT

Skill: Insert New Memory

Purpose: Capture new, durable facts, including procedural knowledge and action sequences, from the current text chunk that are missing in memory.

When to use:
- The text chunk introduces new facts, events, plans, or context worth storing.
- The information is stable and likely useful later.

How to apply:
- Compare against retrieved memories to avoid duplicates.
- Split distinct facts into separate items, including action sequences.
- Keep each item concise and specific.

Constraints:
- Skip trivial, fleeting, or speculative content.
- Do not update or delete existing memories.

Action type: INSERT only.

Evolved Skill on ALFWorld - NOOP

Skill: No Operation

Purpose: Confirm no memory changes are needed for the text chunk.

When to use:
- The text chunk contains no new, corrective, or actionable information.

Constraints:
- Emit NOOP only if none of the selected skills produce actions.

Action type: NOOP only.

Evolved Skill on ALFWorld - TRACK_OBJECT_LOCATION

Skill: Track Object Location

Purpose: Explicitly track the location and state of an object necessary for task completion.

When to use:
- The text chunk mentions an object's location or state.
- The object's location or state is crucial for future task steps.

How to apply:
- Identify the object, its location, and relevant state from the text chunk.
- Create a new memory item with the object-location-state triplet.

Constraints:
- Only track locations and states of objects relevant to the task.
- Update existing location memories if new information is provided.

Action type: INSERT only.

Evolved Skill on ALFWorld - TRACK_OBJECT_MOVEMENTS

Skill: Track Object Movements

Purpose: Track movements of objects necessary for task completion.

When to use:
- The text chunk mentions an object's movement.
- The object's movement is crucial for future task steps.

How to apply:
- Identify the object and its movement from the text chunk.
- Create a new memory item with the object-movement pair.

Constraints:
- Only track movements of objects relevant to the task.
- Update existing movement memories if new information is provided.

Action type: INSERT only.

Evolved Skill on ALFWorld - UPDATE

Skill: Update Existing Memory

Purpose: Revise a retrieved memory with new or corrected information from the text chunk.

When to use:
- The text chunk clarifies, corrects, or extends a retrieved memory.

How to apply:
- Select the best matching memory item.
- Merge new details into a single updated item.
- Preserve accurate details that still hold.

Constraints:
- Do not create new memories.
- Do not delete items.

Action type: UPDATE only.

C. 提示词

LoCoMo Answer Prompt

Based on the above context, write an answer in the form of a short phrase for the following question. Answer with exact words from the context whenever possible.

Question: {} Short answer:

LongMemEval Answer Prompt

I will give you several history chats between you and a user. Please answer the question based on the relevant chat history.
History Chats: {}
Current Date: {}
Question: {}
Short Answer:

HotpotQA Answer Prompt

Based on the following context, answer the question. The question may require reasoning across multiple pieces of information.
{context}
Question: {question}

Instructions:
- Read the context carefully and identify relevant information
- If the answer can be found in the context, provide a short, precise answer
- Output your answer within <answer></answer>tags

<answer>your answer here</answer>

ALFWorld Env Interaction Prompt

You are controlling a text-based ALFWorld environment. Your job: choose the NEXT action as ONE text command. Output ONLY the command string, with no extra text. You MUST choose an action from the admissible actions list and copy it EXACTLY.

Goal: {goal}
Retrieved procedural tips (optional, short & actionable): {retrieved_memory}
Interaction history so far (most recent info matters most): {history}
Admissible actions (choose exactly ONE and copy it verbatim): {actions}

Now output exactly one line: the chosen action (must match one item above).

Executor Prompt

You are a memory management executor. Apply the selected skills to the input text chunk and retrieved memories, then output memory actions.

Input Text Chunk: {session_text}
Retrieved Memories (0-based index): {mem_text}
Selected Skills: {skills_text}

Guidelines:
- Apply any skill as needed; a skill may be used multiple times.
- Read the input text chunk carefully line by line and apply any skill as needed.
- Only use action types supported by the selected skills.
- MEMORY_INDEX is 0-based and must reference the retrieved memories list.
- Output only action blocks in the format below.
- Do not include explanations or REASONING lines.

Output format (repeat as needed). Use ONE block per action and separate blocks with a blank line:

INSERT block:
ACTION: INSERT
MEMORY_ITEM: [concise but complete summary with essential details]

UPDATE block:
ACTION: UPDATE
MEMORY_INDEX: [0-based index]
UPDATED_MEMORY: [concise but complete merged summary with essential updates]

DELETE block:
ACTION: DELETE
MEMORY_INDEX: [0-based index]

Designer Analysis Prompt

You are an expert analyst for a memory-augmented QA system. Analyze the failure cases below to identify why the system failed and how the memory management skills should change.

How This System Works
1. **Memory Storage**: The system applies memory management skills to decide what information to store from the text chunk.
2. **Memory Retrieval**: At question time, it retrieves the most relevant memories by semantic similarity.
3. **Answer Generation**: An LLM answers using the retrieved memories.

Failures can occur at any stage:
- **Storage failure**: Important information was never stored (skill missing or misapplied)
- **Retrieval failure**: Relevant memory exists but was not retrieved (embedding mismatch)
- **Memory quality failure**: Memory exists but is too vague or incomplete to answer

Current Memory Management Skills
{operation_bank_description}

Operation Evolution Feedback
{evolution_feedback}

Failure Cases ({num_failure_cases} cases)
{failure_cases_details}

Analysis Instructions
This is round 1 of a reflection loop. Produce a strong initial analysis that can be critiqued and improved.
1. For each case, check whether the retrieved memories contain the answer or the needed evidence.
2. If missing, decide whether it was never stored (storage failure) or stored but too weak (memory quality failure).
3. If the answer is present but not retrieved, label it retrieval failure and avoid changing skills unless the pattern repeats.
4. Group cases into patterns tied to information types, entities, temporal details, or constraints.
5. For each pattern, propose a concrete skill change: add a new skill or refine an existing one to capture missing details.
6. Provide up to {max_changes} recommendations total (use fewer if only one change is justified).

Output Format
Provide your analysis as JSON:
{ "failure_patterns": [{ "pattern_name": "[descriptive name for this failure pattern]", "affected_cases": [list of case numbers, e.g., 1, 3, 5], "root_cause": "[storage_failure|retrieval_failure|memory_quality_failure]", "explanation": "[why this pattern of failures is occurring]", "potential_fix": "[what kind of operation change could address this]" }],

Designer Refinement Prompt (1/2)

Based on the failure analysis, propose a specific improvement to the memory operation bank.

Failure Analysis (from Stage 1)
{analysis_feedback}

Current Operation Bank
{operation_bank_full}

{evolution_feedback}

Your Task
Propose up to {max_changes} improvements based on the analysis:

Option A - Add New Operation: Create a new operation if the analysis shows a capability gap (e.g., certain information types are not being captured).
Option B - Refine Existing Operation: Improve an existing operation's instruction template if the analysis shows it's not working well (e.g., memories are too vague, missing key details).
Option C - No Change: If the failures are due to retrieval issues (not operation issues), or if the current operations are already well-designed.

CRITICAL Requirements
1. instruction_template MUST be a skill-style guide and MUST NOT include context placeholders (the executor injects the text chunk and retrieved memories)
2. instruction_template MUST clearly state purpose, when to use, and constraints
3. instruction_template MUST specify the allowed action type (INSERT or UPDATE only)
4. For new operations, 'update_type' must be either "insert" or "update" (delete and noop operations are not evolved at this time)
5. Only propose operations with update_type "insert" or "update"
6. Avoid labels like "ENHANCED", "ADVANCED", or other marketing adjectives in descriptions or templates; keep phrasing neutral and task-specific
7. Do NOT embed output blocks; the executor handles output formatting and can apply the skill multiple times
8. The number of changes in the list MUST be less than max_changes
9. Do NOT modify the same operation more than once in a single response, and do NOT refine an operation you add in the same response

Example of a Well-Designed Insert Operation
{ "name": "extract_personal_preferences", "description": "Memory management skill for capturing personal preferences and habits mentioned in the text chunk.", "update_type": "insert", "instruction_template": "Skill: Insert Preferences. Purpose: Capture personal preferences, habits, or opinions stated in the text chunk. When to use: The text chunk mentions likes, dislikes, routines, or goals tied to a person. How to apply: Attribute the preference to the correct person. Keep the preference specific and actionable. Constraints: Avoid one-off or ambiguous statements. Action

Designer Refinement Prompt (2/2)

No changes needed:
{ "action": "no_change", "reasoning": "[why the current operations are sufficient]" }

Instruction Template Structure
When writing instruction templates, follow this structure:

Skill: [Short skill name]
Purpose: [What this skill does]
When to use:
- [Trigger 1]
- [Trigger 2]
How to apply:
- [Step 1]
- [Step 2]
Constraints:
- [What to avoid]
Action type: [INSERT only | UPDATE only]

Output ONLY the JSON, no other text.

LLM Judge Prompt

You are an expert judge evaluating the quality of an answer for a QA task. Your goal is to determine whether the model's answer correctly and sufficiently answers the given question.

Read the following information carefully:

[Question]
{question}

[Ground Truth Answers]
{ground_truth}

[Model Answer]
{model_answer}

Your evaluation criteria:
1. Correctness:
- Is the model answer factually consistent with ANY of the correct answers?
- Does it avoid contradictions or introducing false information?

2. Relevance:
- Does the answer address the question directly without unnecessary content?

3. Completeness:
- Does the answer include all essential information needed to fully answer the question?
- Partial answers are allowed but should receive lower scores.

Scoring Rules:
- Score = 1.0 if the answer is fully correct.
- Score = 0.5 if the answer is partially correct but incomplete or slightly inaccurate.
- Score = 0.0 if the answer is incorrect, irrelevant, or contradicts the ground truth.

Output Format (STRICT):
Return your output as a JSON dictionary with two fields:
{ "explanation": "<brief explanation of your reasoning>", "score": <0.0 | 0.5 | 1.0>}

Be concise and objective. Do not include anything outside the JSON.